LECTURE 4

Scientific Software Stack and Running Simulations on HPC

From user access → software environment → Slurm → CPU parallelism → MPI → GPU → scientific applications

Speaker: Prof. Dr. Tassadaq Hussain

Quantum ESPRESSO | OpenFOAM | GROMACS | Slurm

Core question: “I have a scientific model. How do I actually run it on the Namal HPC cluster?”

Lecture Roadmap

Start with the software stack, then progressively move from one core to real scientific applications.

1	**STACK**	What software sits between the scientist and the hardware?
2	**ACCESS**	Login, shared files, modules and the user environment
3	**EXECUTION**	Slurm: request resources, submit, monitor and collect output
4	**PARALLELISM**	Serial → OpenMP → MPI → GPU
5	**APPLICATIONS**	QE → OpenFOAM → GROMACS
6	**HANDS-ON**	Run small jobs, verify, compare and scale carefully

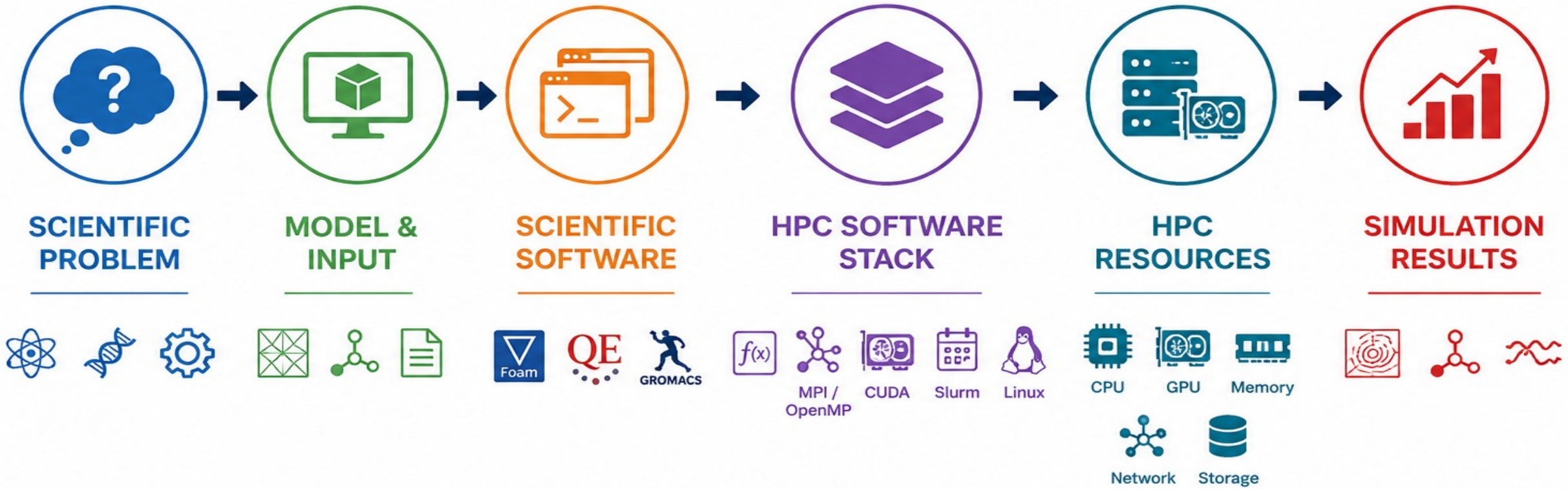
1

HPC Software Stack Echo-System

Understand what each software layer does before running any scientific application.

HPC Software Ecosystem for Modeling & Simulation

The complete set of software components that takes a scientific model from input to computation and finally to results on HPC hardware.



HOW MODELING & SIMULATION WORKS (THE BIG PICTURE)

A REAL PROBLEM IS CONVERTED INTO MATHEMATICS, EXPLORED UNDER MANY SCENARIOS, AND EXECUTED ON POWERFUL COMPUTERS TO GET USEFUL RESULTS.

1 REAL-WORLD PROBLEM

What we want to understand or solve

EXAMPLE PROBLEMS

- Weather Forecast
- Flood Prediction
- Pandemic Spread
- Wind Turbine Design
- ...

GOAL

Make better decisions, save lives, reduce cost

2 BUILD A MATHEMATICAL MODEL

Represent the real system using mathematics

WHAT WE CREATE

- Domain (Where)
- Variables (What)
- Assumptions & Parameters
- Physical Laws & Equations

OUTPUT

A mathematical model of the real system

3 MATHEMATICAL EQUATIONS

The model is written as equations

EXAMPLE: GENERAL FORM (NO COMPLEX MATH HERE)

Change in State = Physics (Processes) + Interaction with Neighbors + External Forcing

We solve these equations for every grid cell / particle over time.

RESULT

A solvable set of equations for the model

4 CHOOSE SCENARIOS

Define different conditions ("what-if" questions)

EXAMPLE SCENARIOS

- Scenario 1 (Base Case)
- Scenario 2 (High Rainfall)
- Scenario 3 (High Temperature)
- ...
- Scenario N (Different Settings)

OUTPUT

Many cases to explore the future

5 RUN THE SIMULATION

Convert equations to numbers and compute over time

HOW IT IS DONE

Divide domain into millions of cells → Apply equations at each cell → Advance one time step → Repeat for thousands to millions of steps → Store all results

OUTPUT

Simulated data (temperature, pressure, flow, levels, etc.)

6 EXECUTE ON COMPUTERS

Use HPC to solve huge computations fast

WHERE & HOW WE EXECUTE

- Code (Program)
- HPC / Supercomputer (Thousands of CPUs / GPUs)
- Parallel Execution (Solve many cells together)
- High Performance (ExaFLOPS Scale)

OUTPUT

Fast results for analysis and decisions

SUMMARY: REAL WORLD → MATHEMATICAL MODEL → EQUATIONS → SCENARIOS → SIMULATION → HPC → RESULTS → DECISIONS

ESTIMATING π USING MONTE CARLO SIMULATION

A simple example of how a **model**, **math** and **computation** work together.

1 WHY A MODEL IS REQUIRED

We want to estimate π (≈ 3.14) **without knowing its exact value.**

Why model?

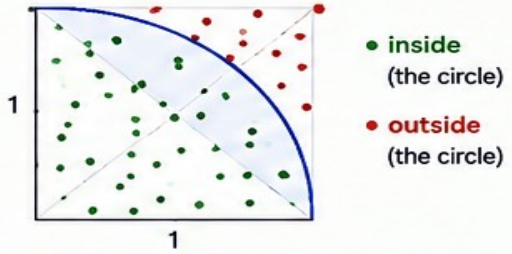
- Real problems are complex.
- We use a simple model to represent the problem.
- The model allows us to estimate π using random points and math.

Goal

Estimate π as close as possible using computation.

2 BUILD THE MODEL

The model: Random points in a 1 × 1 square with a quarter circle (radius = 1).



Mathematical Model

For each random point (x, y):

If $x^2 + y^2 \leq 1$ → inside circle

Else → outside circle

$\pi \approx 4 \times (\text{Inside Points}) / (\text{Total Points})$

Computational Work (per point)

$x^2 + y^2$ → 2 multiplications + 1 addition

= 3 FLOPs (basic arithmetic)

3 EXECUTE MODEL (ITERATION)

Do the calculation for one point.

Example Iteration

Random point:

$x = 0.60, \quad y = 0.50$

Compute:

$x^2 + y^2 = (0.60)^2 + (0.50)^2$

$= 0.36 + 0.25 = 0.61$

Check:

$0.61 \leq 1$ → **INSIDE**

Update Counters

Inside Points = Inside Points + 1

Total Points = Total Points + 1

Work per iteration

≈ 3 FLOPs (math) + small overhead

(compare + random numbers)

4 REPEAT, EXECUTE & UPDATE VALUES

Repeat the same steps for N points. **With more points, the estimate becomes closer to the true π.**

Example Progress

Total Points (N)	Inside Points	Estimated π
100	78	3.1200
1,000	782	3.1280
10,000	7,815	3.1260
100,000	78,432	3.1373
1,000,000	785,148	**3.1406**

Update Rule After Each Iteration

π estimate = 4 × (Inside Points) / (Total Points)

(Values update automatically)

More Iterations → Better Accuracy
More Computation → Better Estimate

What we achieve:

A close estimate of π using simple math, repeated many times.

Build Model
(Simple representation of the problem)

→

Execute (Iterate)
Do math for one point
Update counts

→

Repeat Many Times
More points
More computation

→

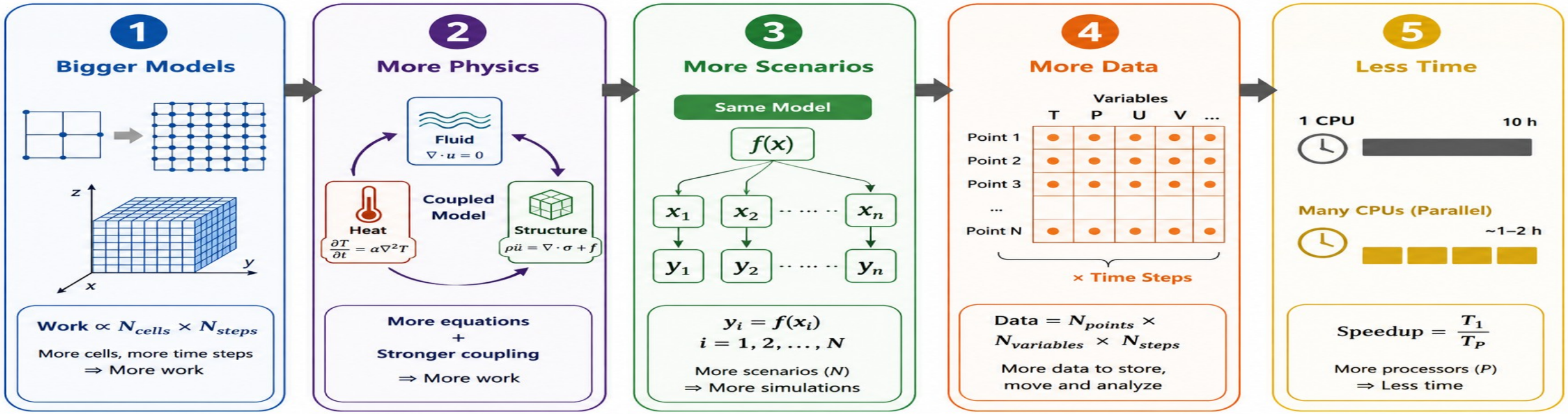
Better Estimate
π converges to
≈ 3.14159

Key Takeaway

A model + math + repetition on computers turns small calculations into accurate answers.

HPC as the Engine of Modern Modeling and Simulation

More resolution, more physics, more scenarios, less waiting



- Finer meshes
- More atoms
- Larger domains
- Longer time horizons

Multiphysics
Coupled models
Turbulence
Quantum effects

Parameter sweeps
Uncertainty
Optimization
Ensembles

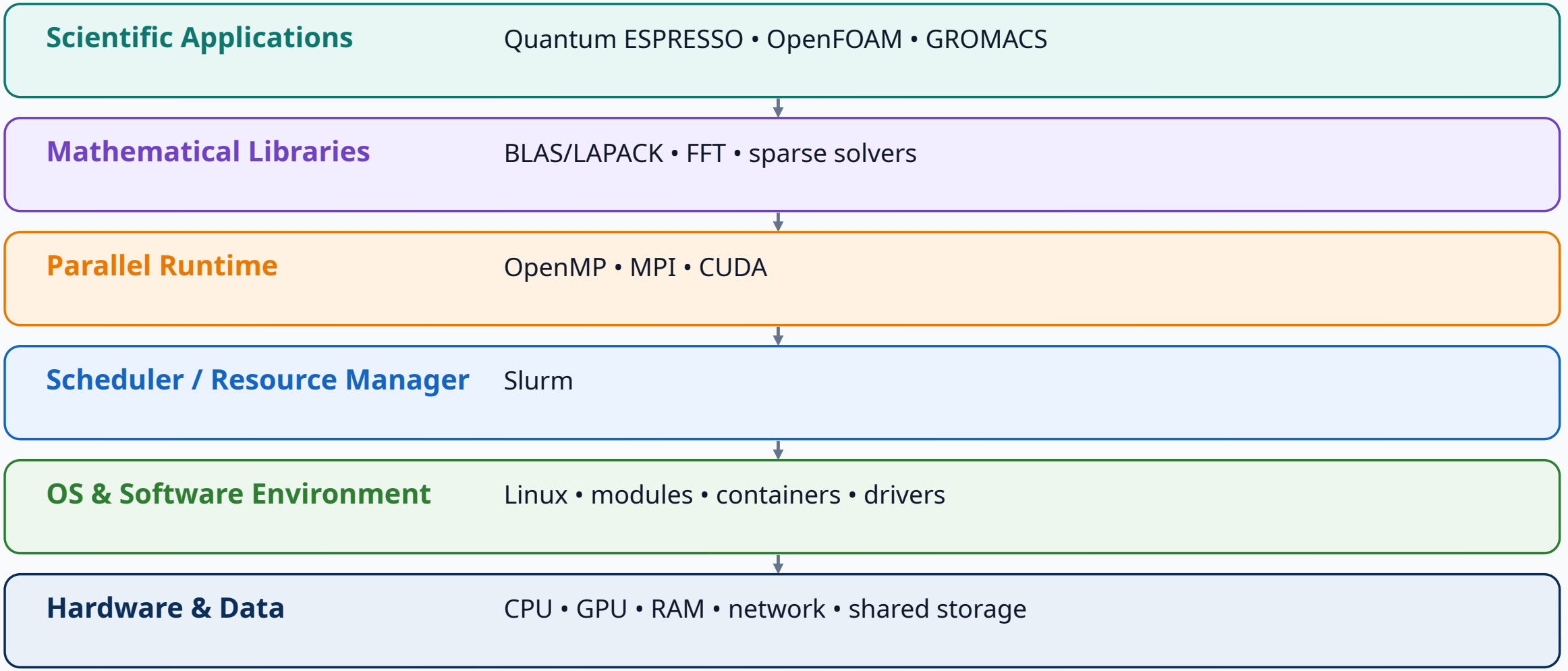
Satellite data
Sensor streams
Trajectories
AI training

Hours → minutes
Days → hours
Months → days

HPC changes the scientific question from “Can we run it?” to “What can we learn from it?”

PakHPC-Sim: Software Stack for Modeling & Simulation

A scientific application sits on top of multiple software layers that connect the model to the hardware.



User usually interacts most with: Application + Modules + Slurm

What Does Each Layer Do?

Keep the mental model simple: each layer has one primary responsibility.

APPLICATION

Implements the scientific model and numerical algorithm.

LIBRARIES

Perform optimized mathematics such as FFTs and linear algebra.

OPENMP / MPI / CUDA

Expose parallelism across CPU cores, nodes and GPUs.

SLURM

Decides when and where resources are allocated to the job.

LINUX + MODULES

Provide the runtime environment, drivers and software versions.

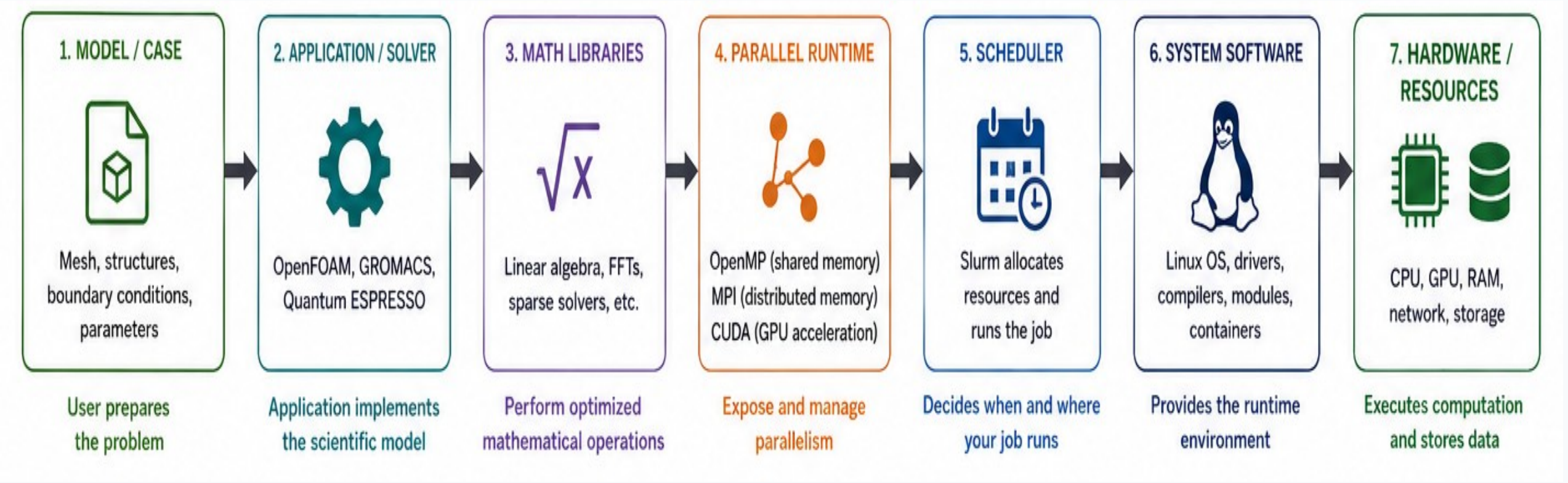
HARDWARE

Executes the computation and moves/stores the data.

Key idea **The scientist describes the problem; the software stack converts it into parallel numerical work on cluster resources.**

How the Stack Works Together

Example path for an OpenFOAM simulation. The same idea applies to GROMACS and Quantum ESPRESSO.



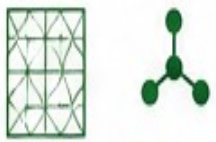
Simulation experts do not write MPI/OpenMP/CUDA code when using established scientific software — the application already contains the parallel implementation.

What Each Layer Provides

1. MODEL / CASE

- User defines physics
- Geometry / mesh
- Materials & BCs
- Initial & boundary conditions

Examples



2. APPLICATION / SOLVER

- Implements numerical algorithms
- Solves governing equations
- Reads input and writes output

Examples

OpenFOAM®

GROMACS
FAST. FLEXIBLE. FREE.

Q: QUANTUM ESPRESSO

3. MATH LIBRARIES

- Highly optimized building blocks for computation
- Reduce development effort and increase performance

What they do

BLAS — Basic Linear Algebra (vectors, matrices)

LAPACK — Linear Algebra (solvers, factorizations)

FFT — Fast Fourier Transforms

Sparse Libraries — Sparse matrix operations

4. PARALLEL RUNTIME

- Enables parallel execution on CPUs and GPUs
- Scales across cores, nodes and accelerators

Tools

OpenMP — Shared-memory parallelism (multi-core)

MPI — Distributed-memory parallelism (multi-node)

CUDA — GPU acceleration

5. SCHEDULER

- Allocates nodes, CPUs, GPUs, memory
- Queues and runs jobs
- Monitors and accounts usage

Tool

slurm
workload manager

- sbatch – submit job
- squeue – view jobs
- sacct – accounting
- scontrol – manage jobs

6. SYSTEM SOFTWARE

- Linux operating system
- Device drivers
- Compilers & toolchains
- Environment Modules
- Containers (Apptainer)

Components

Compilers (GCC, LLVM, Intel)

Modules (manage software)

Containers (Apptainer/Singularity)

7. HARDWARE / RESOURCES

- Compute: CPU & GPU
- Memory: RAM / HBM
- Network: high-speed interconnect
- Storage: shared filesystem

Resources

CPU GPU Memory

Network Storage

2

User Access and Software Environment

Before running science: know where you log in, where files live, and how software is loaded.

The Anatomy of an HPC System

Hardware alone does not create useful supercomputing

A scientific simulation application framework contains implemented mathematical/physical models, numerical algorithms, and solvers. The user describes the real problem using the software's available models, provides the required inputs and conditions, and the software converts that setup into numerical computations that are executed on the CPU/GPU/HPC system.

Scientific Applications — GROMACS • Quantum ESPRESSO • OpenFOAM • AI workloads

Parallel Programming & Libraries — MPI • OpenMP • CUDA • math libraries

Resource Management — Slurm • queues • jobs • accounting

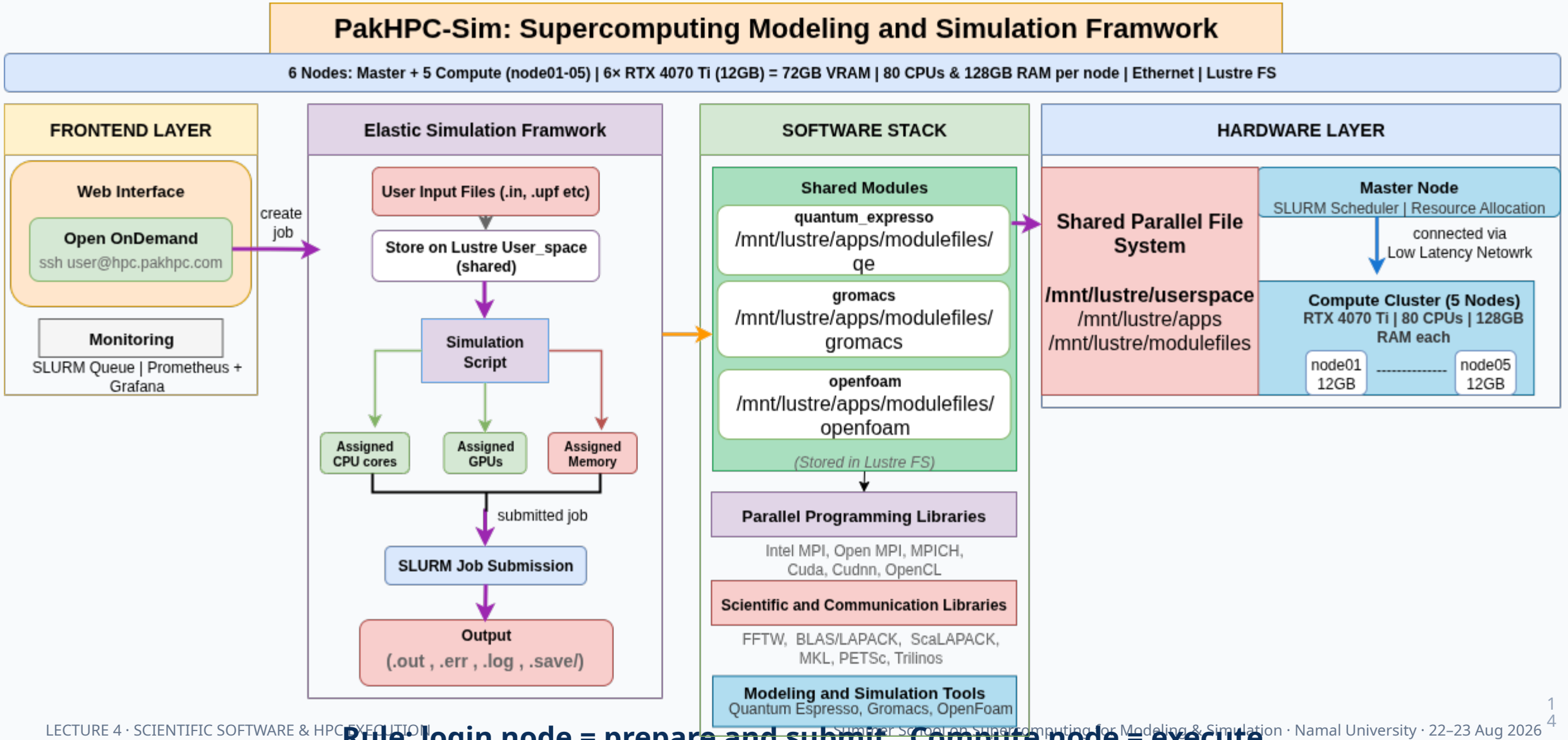
System Software — Linux • compilers • drivers • containers

Hardware — CPU nodes • GPU nodes • memory • network • storage

Useful HPC = balanced system + parallel software + people who know how to use it.

How a User Reaches PakHPC-Sim

Heavy computation runs on compute nodes — not on the login node.



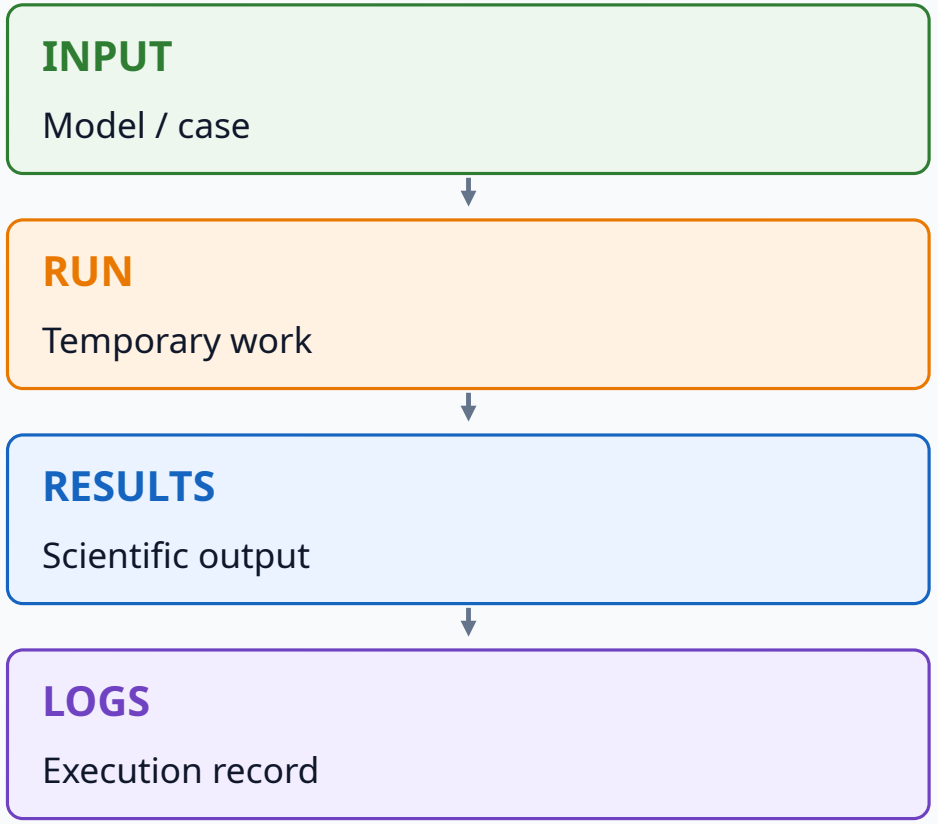
Rule: login node = prepare and submit. Compute node = execute.

Where Are My Files?

Use a predictable project structure so simulations are easier to reproduce and debug.

Example project directory

```
project/
├── input/        # model, mesh, structures
├── scripts/      # Slurm scripts
├── run/          # execution workspace
├── results/      # final scientific output
└── logs/         # stdout / stderr
```



Shared storage makes the same project visible from the login node and allocated compute nodes.

Software Environments: Modules

Modules activate the correct compiler, library and application environment without changing the base OS.

Beginner commands

```
module avail
module load <application>
module list
which <application>
<application> --version
```

3 Verify

module list / which

4 Run

application command

Examples on PakHPC-Sim

Quantum ESPRESSO • OpenFOAM • GROMACS

First Interaction with the Cluster

Before submitting science, confirm the environment and cluster state.

pwd

Where am I?

ls

What files are here?

module avail

What software is available?

sinfo

What cluster resources are available?

Goal of this step: understand the environment — do not start by requesting many CPUs or GPUs.

3

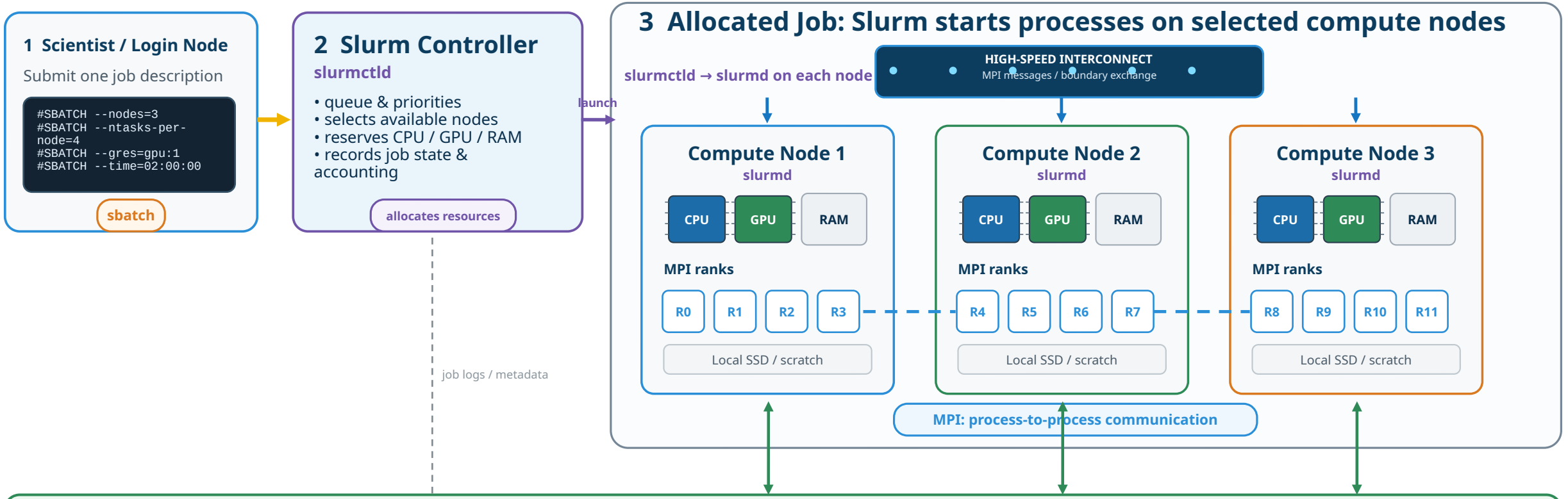
Running a Job with Slurm

Start with a tiny job. Learn the execution path before running a real simulation.

How Slurm Turns a Cluster into a Shared Computing Facility

SOFTWARE

Users request resources; Slurm decides when and where the job runs



SLURM = WHERE & WHEN resources run | MPI = HOW parallel processes communicate | PARALLEL FS = WHERE shared data lives

Your First Slurm Job

A minimal job proves that your command is executing on an allocated compute node.

hello_hpc.sh

```
#!/bin/bash
#SBATCH --job-name=hello_hpc
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --time=00:05:00
#SBATCH --output=hello.%j.out

hostname
nproc
free -h
```

1 Submit

sbatch hello_hpc.sh

2 Monitor

squeue -u $USER

3 Complete

Job leaves queue

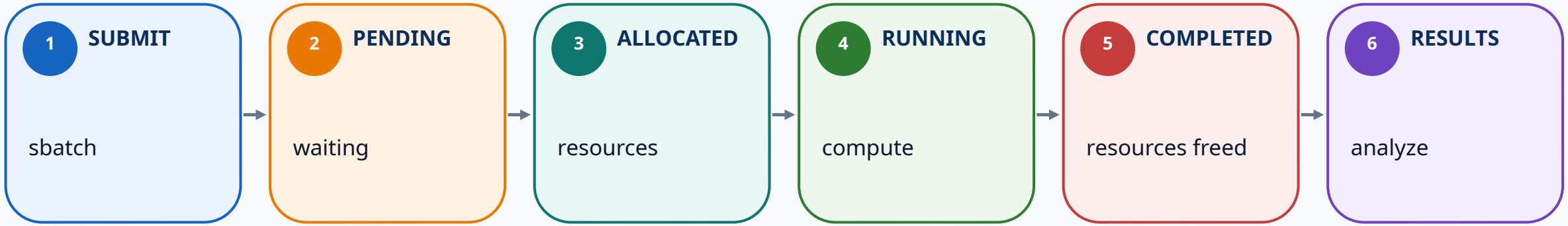
4 Inspect

cat hello.<id>.out

Concept learned: the scheduler moved the work from the login node to a compute node.

Slurm Job Lifecycle

For a beginner, these six states are enough to understand the scheduler.



Slurm = WHERE and WHEN the job runs. The scientific application = WHAT computation is performed.

One Generic Slurm Script for Scientific Applications

Change the resource request and the final application command — the structure stays almost the same.

Reusable template

```
#!/bin/bash
#SBATCH --job-name=my_sim
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --cpus-per-task=1
#SBATCH --time=00:30:00
#SBATCH --output=job.%j.out

module load <application>
srun <application-command>
```

WHAT?	Application / command
HOW MUCH?	CPUs • GPU • memory
WHERE?	Nodes / partition
HOW LONG?	Wall time
RUN	srun / application command

One Generic Slurm Script for Scientific Applications

Change the resource request and the final application command — the structure stays almost the same.

Reusable template

```
#!/bin/bash
#SBATCH --job-name=my_sim
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --cpus-per-task=1
#SBATCH --time=00:30:00
#SBATCH --output=job.%j.out

module load <application>
srun <application-command>
```

WHAT?	Application / command
HOW MUCH?	CPUs • GPU • memory
WHERE?	Nodes / partition
HOW LONG?	Wall time
RUN	srun / application command

Parallel Computing: Why Scientific Problems Need Many Cores

HPC ARCHITECTURE

Split the work so that multiple processing elements make progress together

1 REAL-WORLD PROBLEM

Define objective, physics, and constraints.

- Geometry
- Materials
- Boundary conditions
- Loads / Inputs

2 MATHEMATICAL MODEL

Governing equations represent the physics.

Example (Fluid Flow)

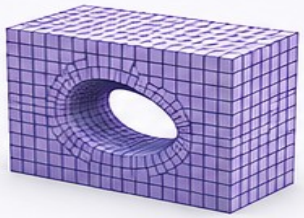
$$\nabla \cdot \boldsymbol{u} = 0$$

$$\rho\left(\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{u} \cdot \nabla \boldsymbol{u}\right) = -\nabla p + \nabla \cdot \tau + \boldsymbol{f}$$

- PDEs
- Constitutive models
- Initial / Boundary conditions

3 DISCRETIZATION (MODEL TO MESH)

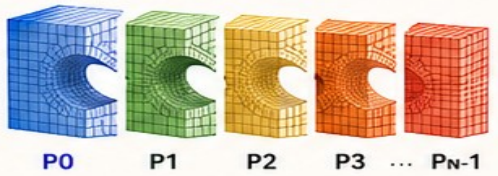
Convert equations to a numerical problem.



- Mesh generation
- Discretize equations
- Assemble algebraic system $Ax = b$

4 PARALLEL DECOMPOSITION (DISTRIBUTED DATA)

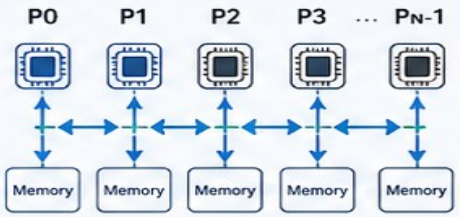
Split the domain across many processors.



- Domain decomposition
- Local subdomain on each rank
- Communicate at boundaries

5 DISTRIBUTED EXECUTION (PARALLEL COMPUTING)

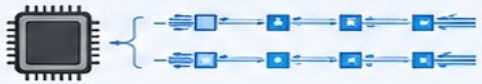
Processors solve their part simultaneously.



- Each process solves local equations
- Exchange boundary data
- Iterate to convergence

PARALLEL PROGRAMMING LAYER

Shared Memory (Node Level)
OpenMP / Pthreads



Distributed Memory (Cluster Level)
MPI

Accelerators (Node Level)
CUDA / HIP / OpenCL

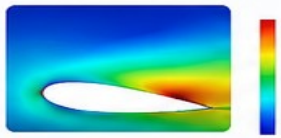
HPC INFRASTRUCTURE

Compute Nodes (CPU / GPU)

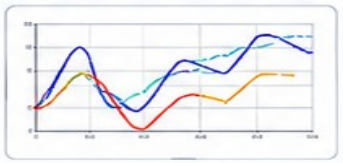
High-Speed Interconnect

Parallel File System (shared storage)

SIMULATION RESULTS & INSIGHTS



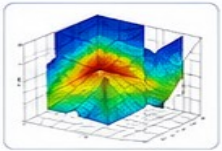
Field Results (Pressure, Velocity, Temperature, etc.)



Probes / Time History

Lift	123.4 N
Drag	12.7 N
Power	4.56 kW

Quantities / Reports



Visualization

Scaling: More Hardware Does Not Automatically Mean More Speed

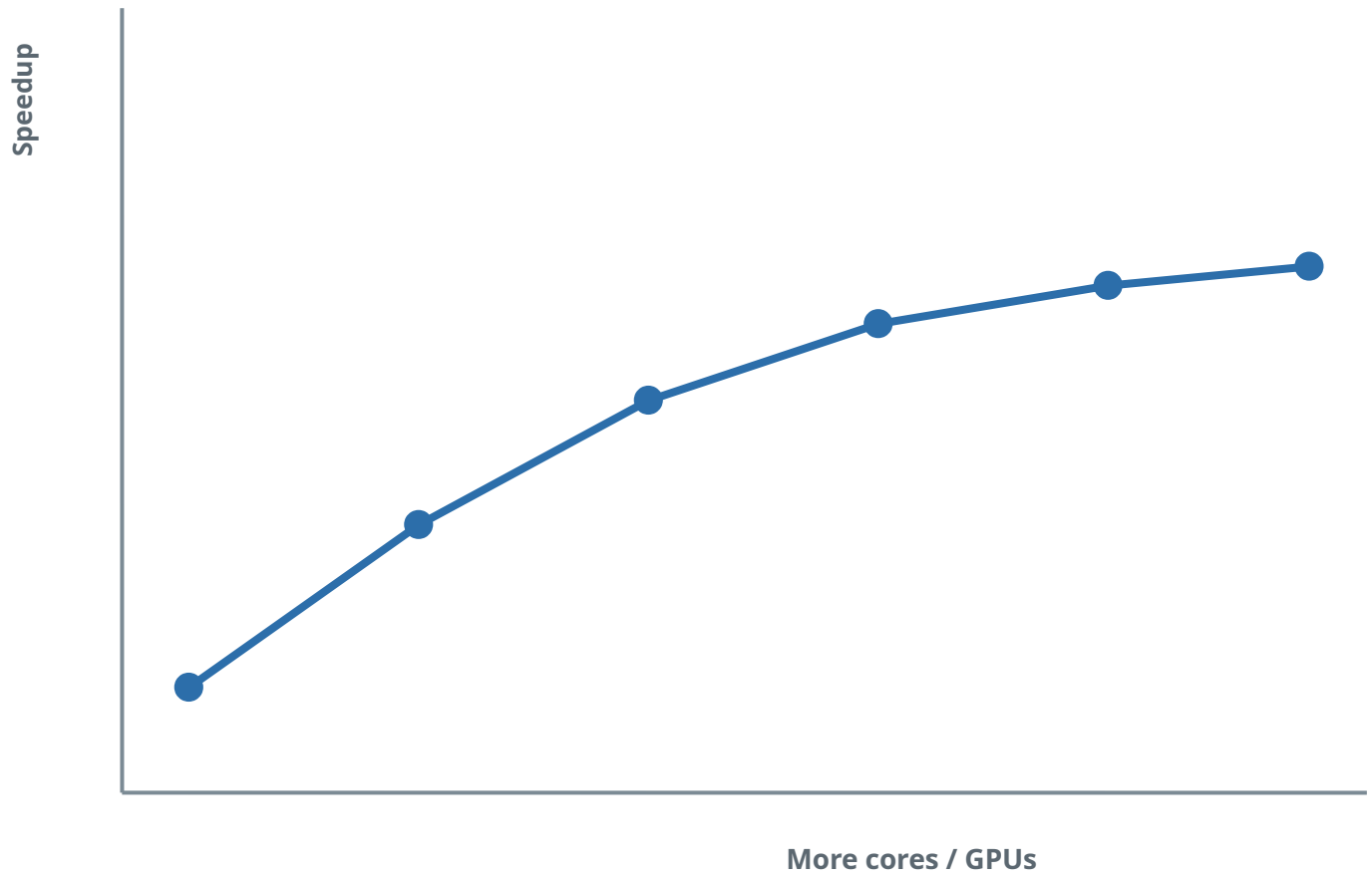
Parallel efficiency determines whether extra resources are useful

Strong Scaling

Fixed problem size
Add cores → reduce wall time
Question: How far can this problem scale?

Weak Scaling

Increase problem size with resources
Question: Can throughput stay efficient?



Communication, synchronization, serial work and memory bandwidth cause diminishing returns.

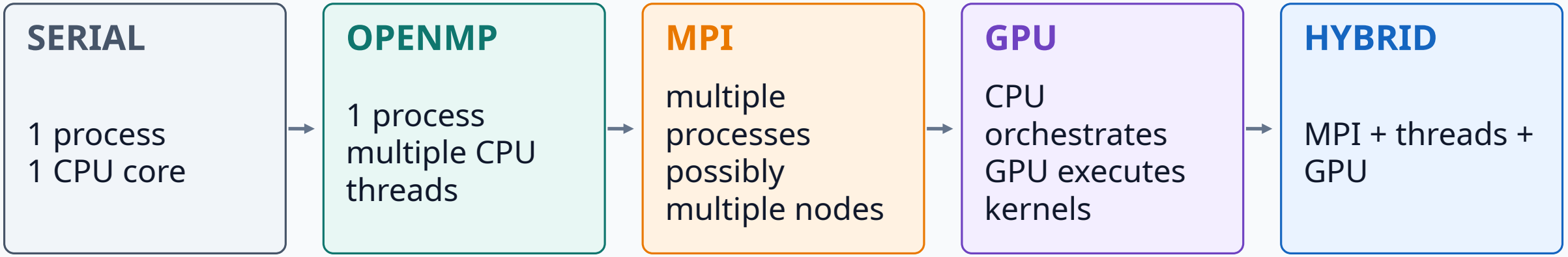
4

From One Core to Parallel Execution

Learn parallelism progressively: serial → OpenMP → MPI → GPU → hybrid.

Parallelism Ladder

Each step exposes a different level of the hardware hierarchy.



Hands-on principle: start at the left. Move right only after the smaller run is correct.

Level 0 — Serial Baseline

Always establish a correct, small baseline before adding parallel resources.

ONE PROCESS

Process 0

CPU Core 0

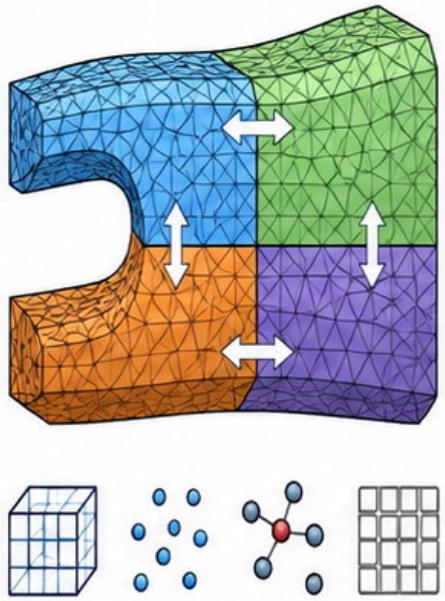
One instruction stream

WHY START HERE?

- Verify the input/model
- Confirm software runs
- Record runtime
- Create a reference for comparison

Correctness first → performance second → scaling third

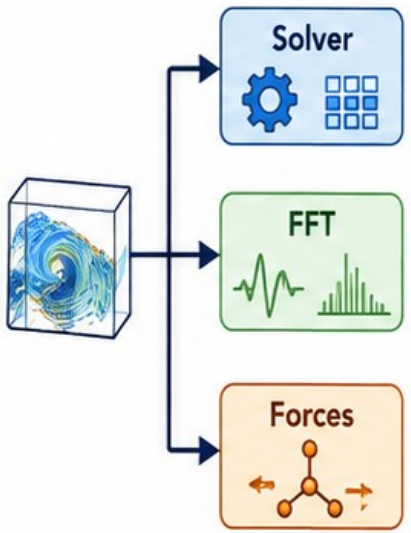
Types of Parallelism in Modeling & Simulation



DOMAIN / DATA

Mesh • particles • atoms • grid

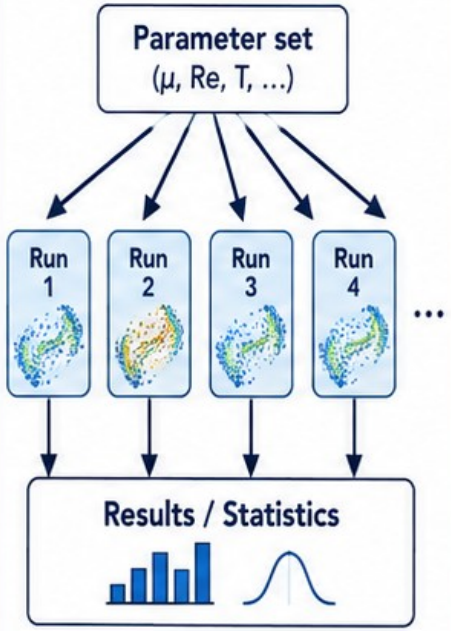
Split CFD mesh



TASK / FUNCTIONAL

Different computational operations

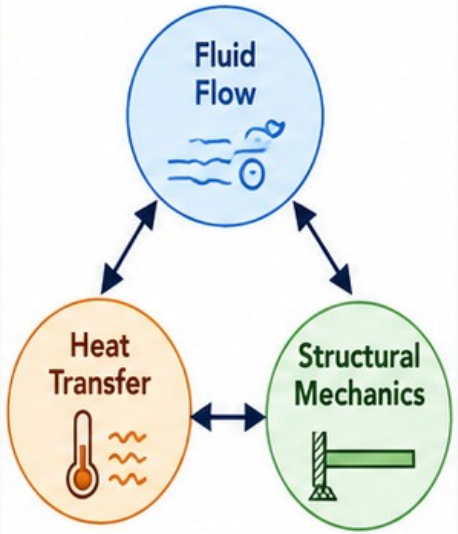
Solver + FFT + forces



SIMULATION / ENSEMBLE

Independent cases

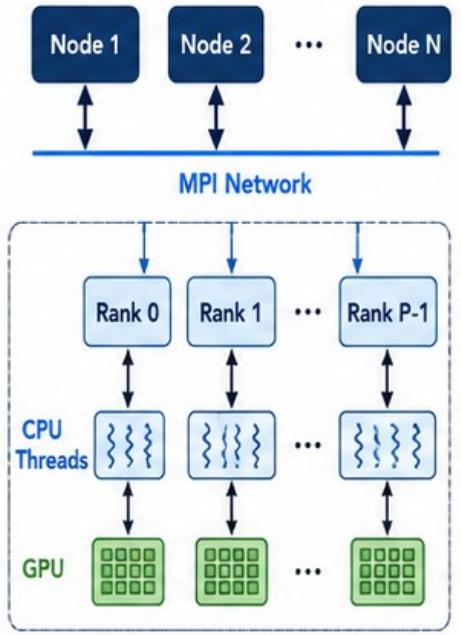
100 parameter combinations



MODEL / COMPONENT

Large / coupled model components

Fluid + thermal + structural

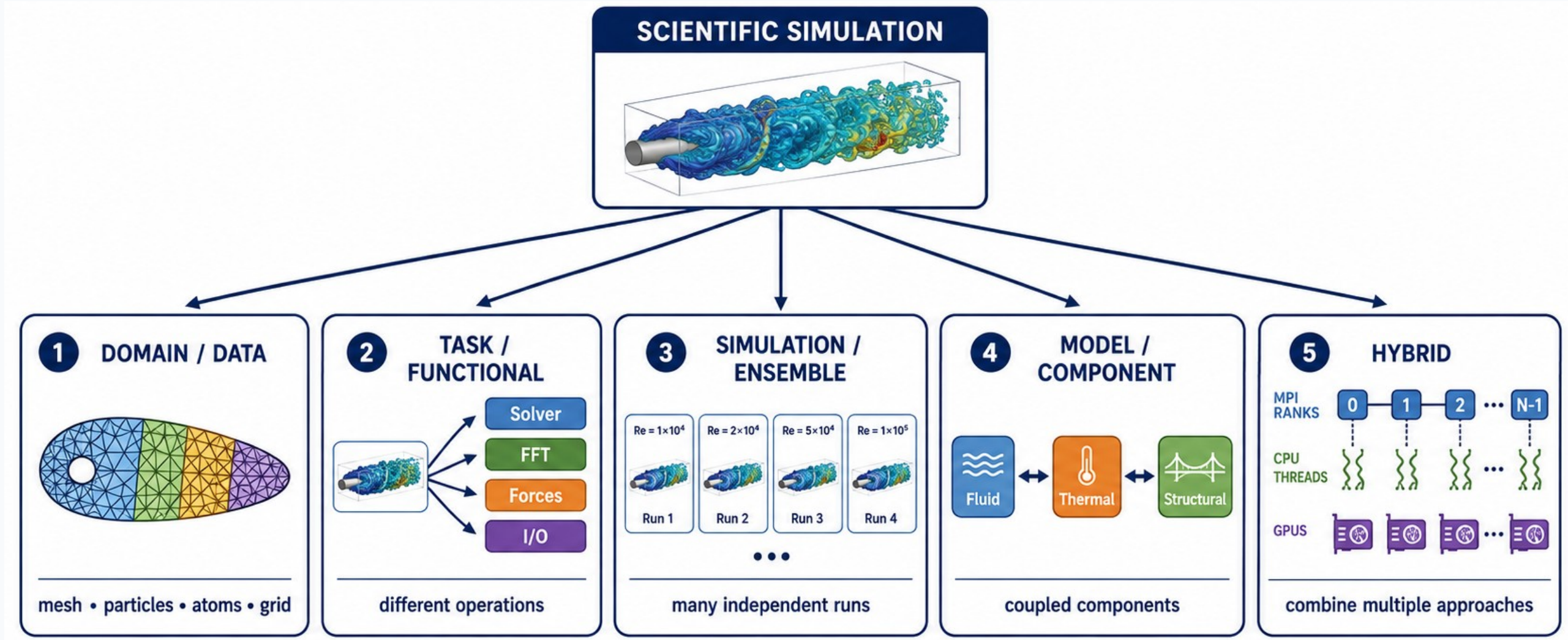


HYBRID

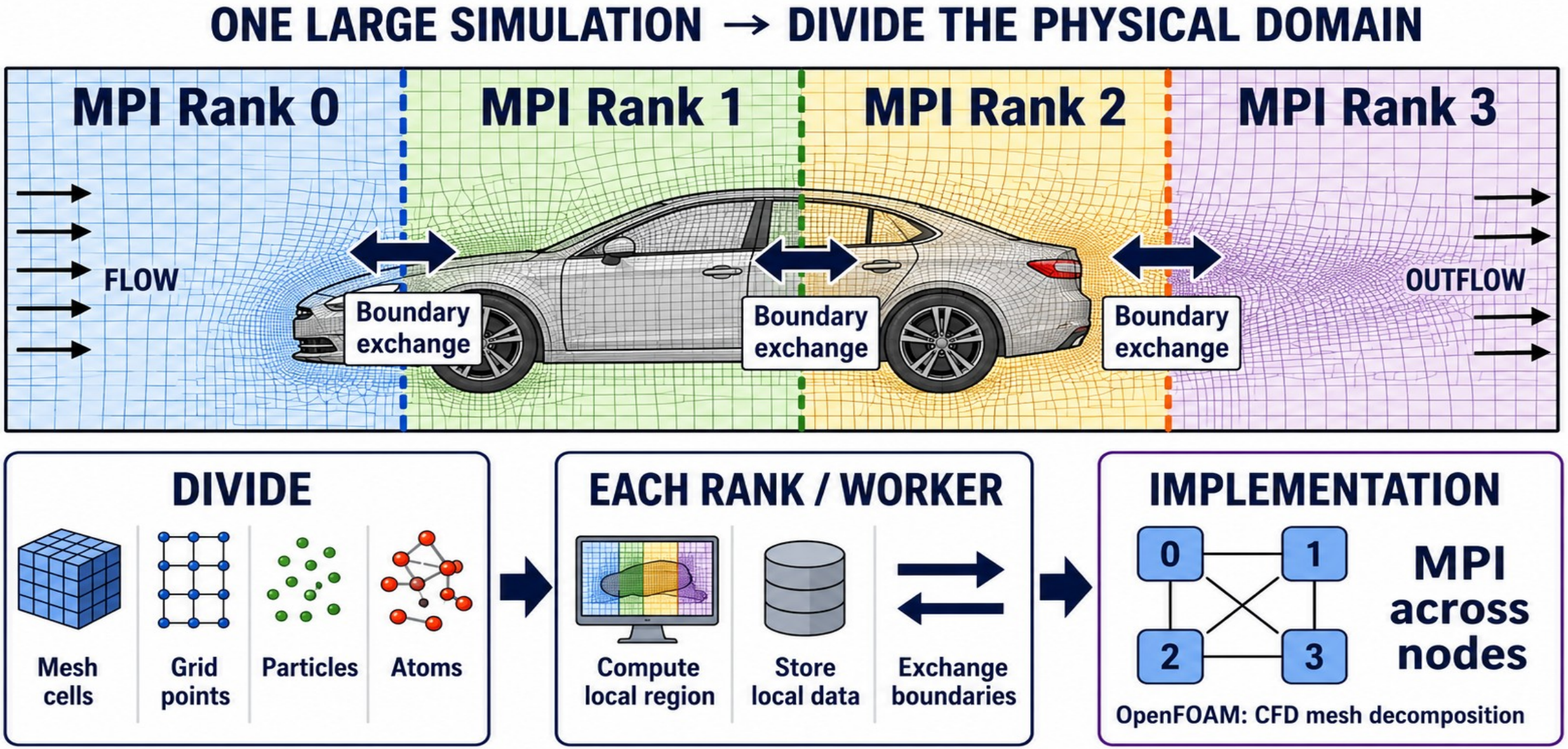
Combination of parallel approaches

MPI + threads + GPU

What Do We Parallelize?



Domain / Data Parallelism



TWO DIFFERENT WAYS TO PARALLELIZE

TASK / FUNCTIONAL PARALLELISM

One simulation contains different operations

ONE SIMULATION

SOLVER

$$\begin{bmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Linear systems, matrix operations

FFT

Fast Fourier Transform

FORCES

Force calculation, interactions

I/O

Input / Output, data writing

DIFFERENT TASKS OPERATE CONCURRENTLY

EXAMPLES

- Matrix operations
- Force calculation
- I/O
- Physics modules

ENSEMBLE / SIMULATION PARALLELISM

Many independent simulations

PARAMETER SET
(e.g., inlet velocity, temperature, material)

RUN 1 (Params 1)

RUN 2 (Params 2)

RUN 3 (Params 3)

...

RUN N (Params N)

RESULTS / STATISTICS

EXAMPLES

- Parameter sweeps
- Monte Carlo
- Optimization
- Uncertainty analysis

TASK PARALLELISM
= divide ONE simulation

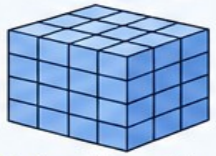
VS

ENSEMBLE PARALLELISM
= run MANY simulations

SCIENTIFIC PROBLEM
Large, complex simulation or analysis

WHAT CAN BE DIVIDED? (PARALLELISM STRATEGIES)

DOMAIN / DATA
Divide the physical domain or data



Mesh, grid, particles, atoms, cells

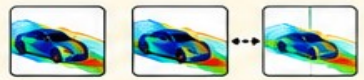
Example: Split CFD mesh

TASK / FUNCTIONAL
Divide different computational tasks

Solver, FFT, forces, matrix ops, I/O, etc.

Example: Solver + FFT + Forces

ENSEMBLE / RUNS
Run many independent simulations



Parameter sweeps, Monte Carlo, optimization

Example: 100 parameter sets

MODEL / COMPONENT
Divide coupled model components

Fluid + thermal + structural, multiphysics coupling

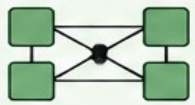
Example: Coupled multiphysics

HOW IS IT EXECUTED? (PARALLEL IMPLEMENTATIONS)

OpenMP (Shared Memory)

Parallelism on multiple cores within a node

MPI (Distributed Memory)



Parallelism across nodes using message passing

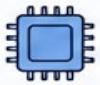
GPU (Accelerators)

Massively parallel execution on GPUs

Hybrid (MPI + OpenMP + GPU)

Combination of MPI, threads and GPUs for maximum performance

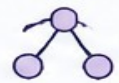
HPC RESOURCES (WHERE IT RUNS)



CPU CORES
Compute on multiple cores

NODES
Multiple compute nodes

GPUS
Accelerated compute for demanding kernels

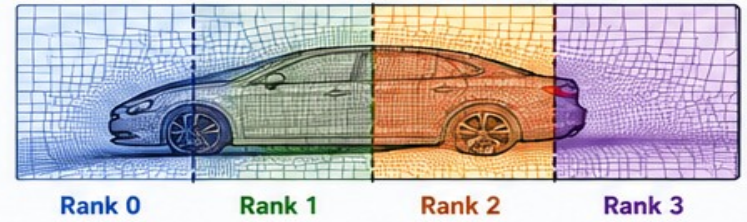


NETWORK
High-speed interconnect for communication

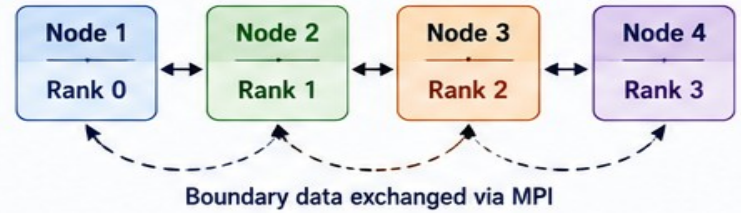
TAKEAWAY: **Domain decomposition** is a simulation strategy.
MPI is a mechanism used to implement it (along with others).

EXECUTION & DISTRIBUTION EXAMPLE (Domain Decomposition with MPI)

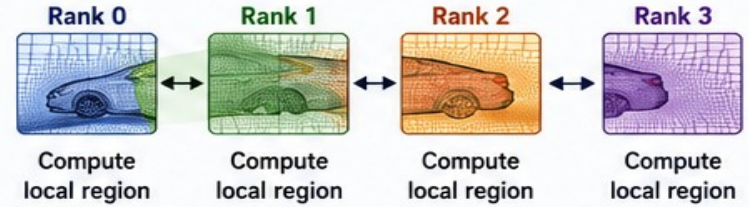
1 DECOMPOSE (Strategy)
Split the domain across MPI ranks



2 DISTRIBUTE (Execution)
MPI ranks on different nodes



3 COMPUTE & COMMUNICATE
Each rank computes its part and exchanges boundary information



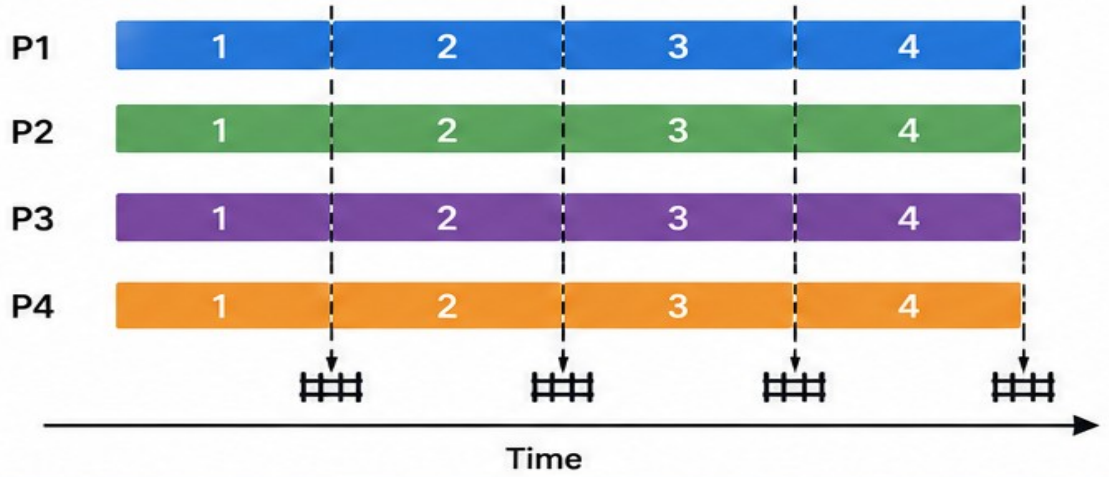
Data / Message Exchange

Boundary Exchange

SYNCHRONOUS vs ASYNCHRONOUS PARALLELISM

SYNCHRONOUS

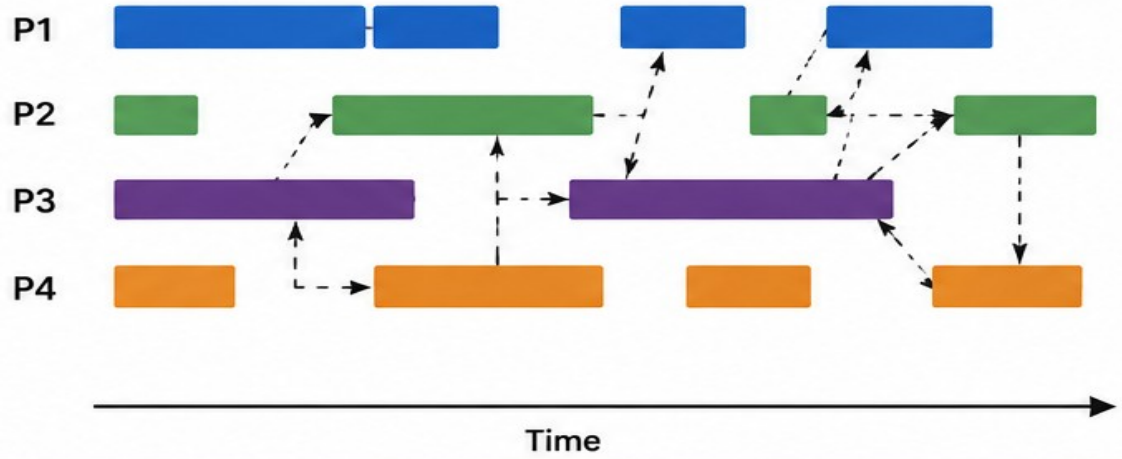
Work in lockstep — wait at barriers

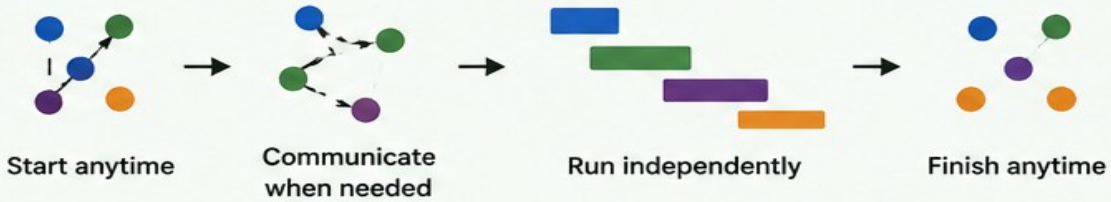


Simple | Predictable | Idle time at barriers

ASYNCHRONOUS

Work independently — no waiting



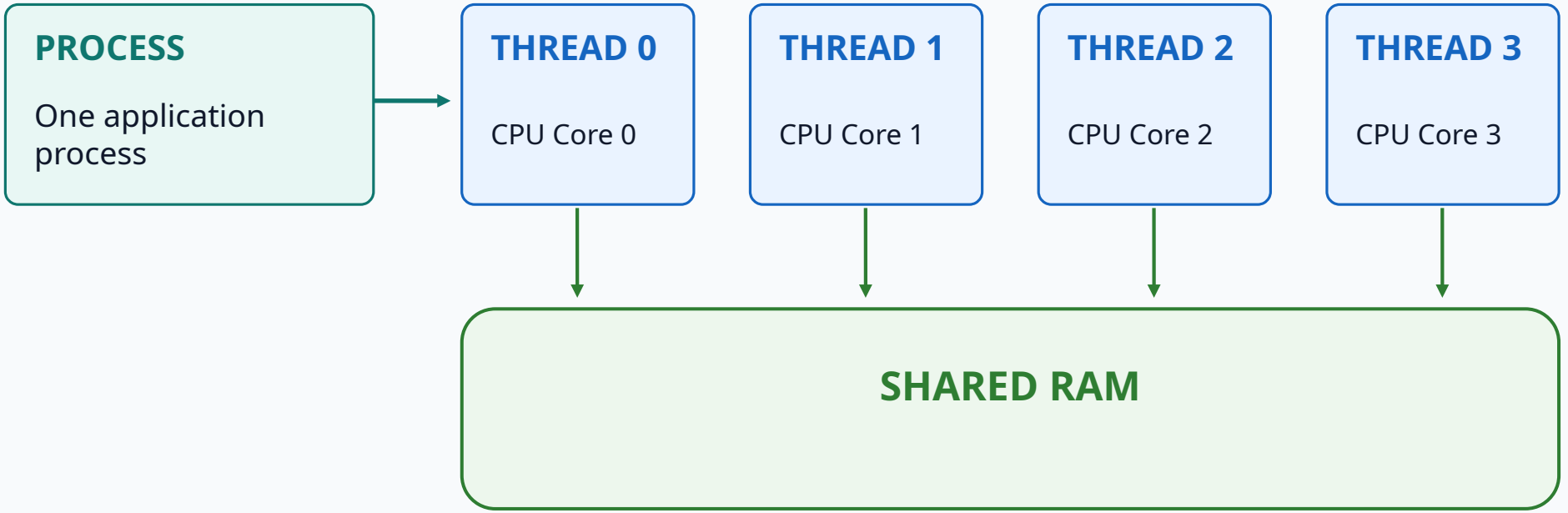


Flexible | Better resource utilization | More complex

Use what fits your problem.
Often: synchronous within phases, asynchronous between phases.

Level 1 — OpenMP: Shared-Memory CPU Parallelism

One process creates multiple threads that work on the same node and share RAM.



Use when: one node has enough memory and the application supports CPU threading.

Hands-On A — Basic OpenMP on One Node

Use a tiny numerical example to observe how one process uses several CPU cores.

OpenMP job idea

```
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4

export OMP_NUM_THREADS=4
srun ./openmp_example
```

4 threads

compare

8 threads

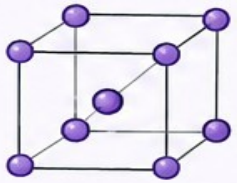
optional

Record runtime — do not assume 4 threads = 4× faster.

Quantum ESPRESSO SCF Workflow

Use a tiny SCF calculation to connect atomic structure → numerical solver → computed energy.

1 STRUCTURE



atoms + cell

→

2 INPUT

all settings for the calculation

→

3 SCF (pw.x)

iterative solution of Kohn–Sham equations

→

4 CONVERGENCE

converged electron density

→

5 RESULT

E = −114.567 Ry

total energy, forces, etc.

INPUT: scf.in file

It tells pw.x what to calculate, how to calculate, and when to stop.

CONTROL

```
calculation,
outdir,
pseudo_dir,
prefix, ...
```

Controls the type of run and where files are written.

SYSTEM

```
ntyp, nat,
ecutwfc,
ecutrho,
occupations, ...
```

Defines the system and numerical accuracy.

ELECTRONS


```
conv_thr,
mixing_beta,
electron_maxstep,
diagonalization, ...
```

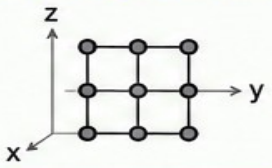
Sets how the electronic SCF is solved and converged.

ATOMIC_SPECIES

```
Si  28.0855  Si.pbe-spn-rrkjus.UPF
O   15.999   O.pbe-n-rrkjus.UPF
```

Lists each element, its mass, and the pseudopotential file.

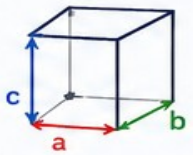
ATOMIC_POSITIONS




```
Si   0.0000 0.0000 0.0000
Si   0.2500 0.2500 0.2500
O    0.2500 0.2500 0.0000
...
```

Gives the position of each atom in the cell (fractional or Cartesian).

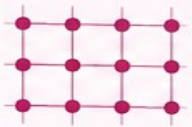
CELL_PARAMETERS



$$\begin{matrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{matrix}$$

Defines the three lattice vectors of the simulation cell.

K_POINTS



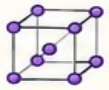
```
Automatic
8 8 8 0 0 0
(Monkhorst–Pack)
```

Specifies how the Brillouin zone is sampled.

PSEUDOPOTENTIALS


```
Si.pbe-spn-rrkjus.UPF
O.pbe-n-rrkjus.UPF
```

Files that describe the interaction of valence electrons with atomic cores.

In one line:  Structure +  scf.in (settings) →  pw.x SCF iterations →  Converged electron density →  Total energy & forces

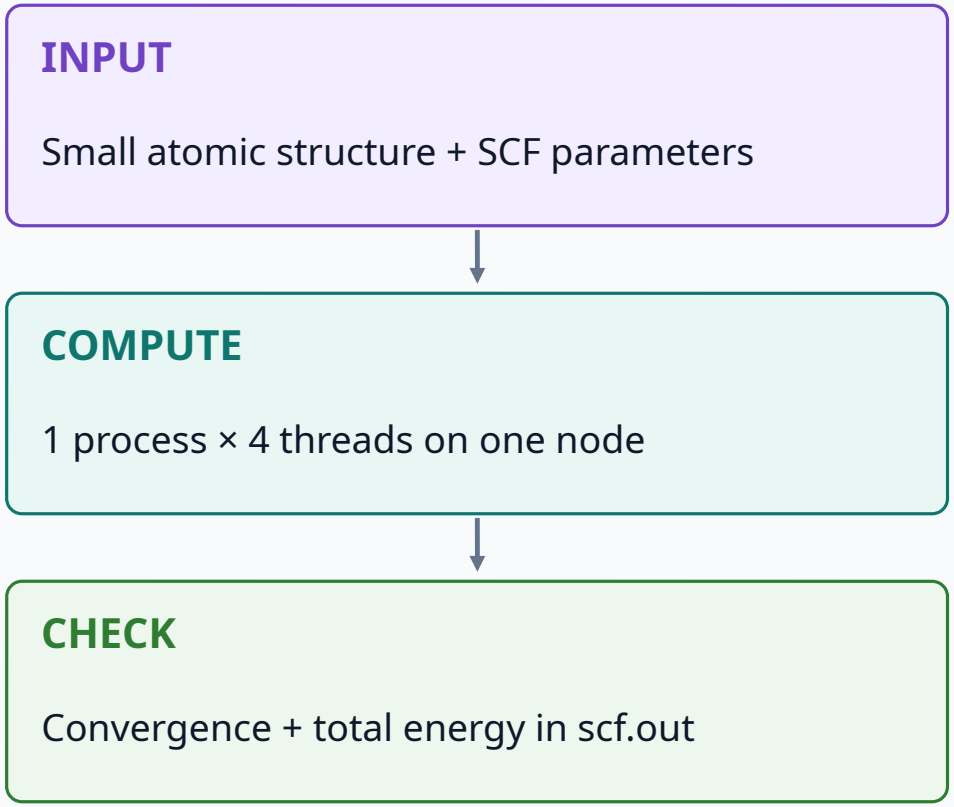
Hands-On B — Quantum ESPRESSO on One Node

Start with one MPI process and a small number of OpenMP threads.

Conceptual example — adapt module name to cluster

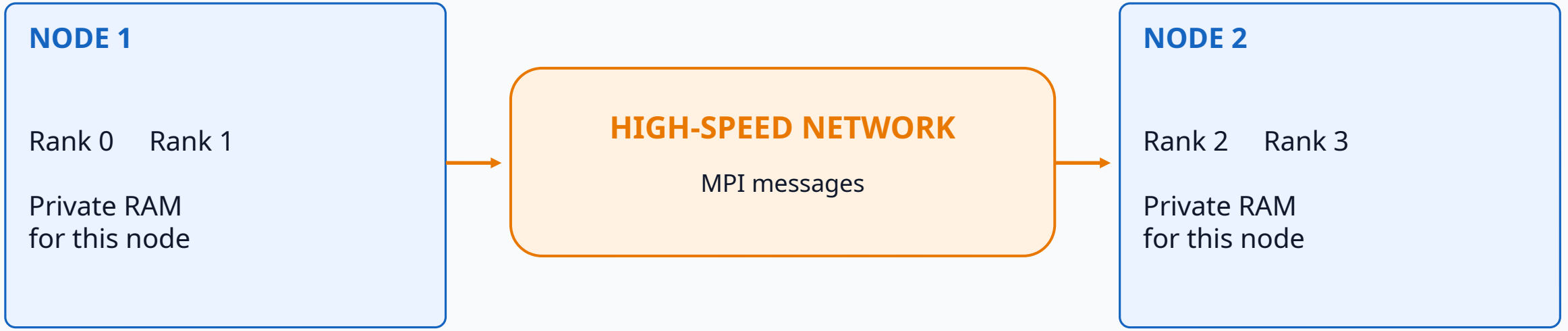
```
#!/bin/bash
#SBATCH --job-name=qe_scf
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --time=00:15:00

module load quantum-espresso
export OMP_NUM_THREADS=4
srun pw.x -in scf.in > scf.out
```



Level 2 — MPI: Distributed-Memory Parallelism

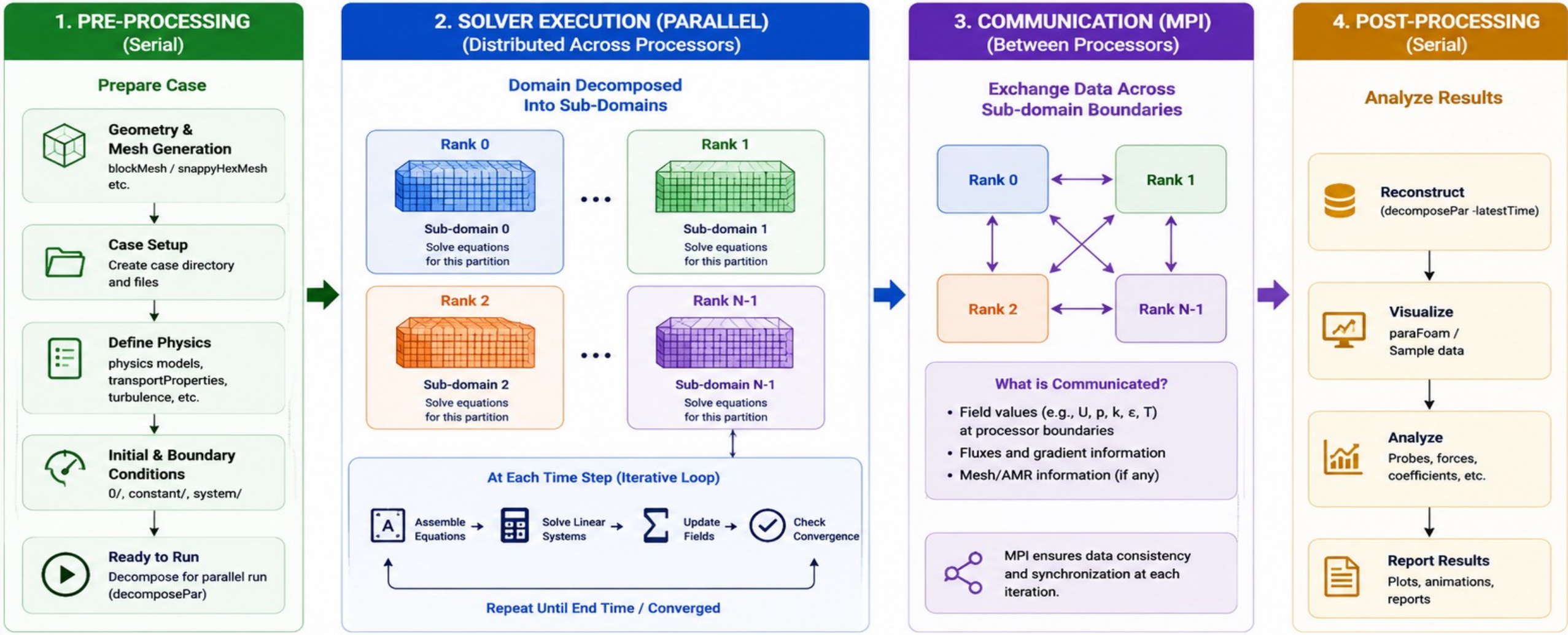
Multiple independent processes (ranks) communicate through messages.



MPI is ideal when the scientific domain can be divided across processes and, if needed, across nodes.

OpenFOAM: MPI Through Domain Decomposition

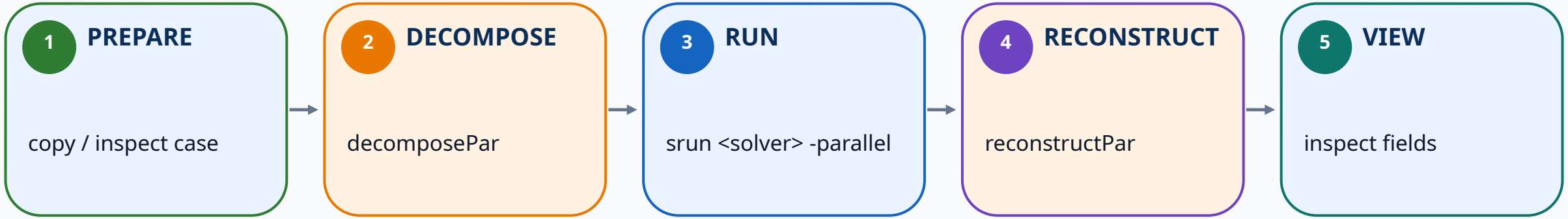
The mesh is split into subdomains. Each MPI rank solves one part and exchanges boundary data.



OpenFOAM already implements MPI — the user chooses the decomposition and process count.

Hands-On C — OpenFOAM Parallel CFD

Use a tiny cavity case so the parallel workflow is visible and fast.



Three commands that expose the MPI workflow

```
decomposePar
srun <solver> -parallel
reconstructPar
```

Goal: understand domain decomposition — not OpenFOAM internals.

OPENFOAM PARALLEL WORKFLOW ON HPC (SLURM + MPI)

Example Case: lid-driven cavity (icoFoam)

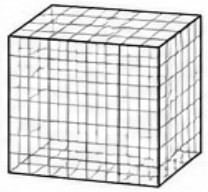
1 GET THE CASE

Copy tutorial case
(lid-driven cavity)

`cavity_mpi/`

2 CREATE MESH

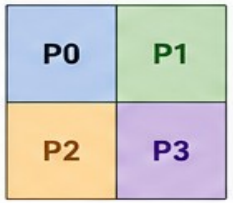
Generate mesh
for the CFD domain



blockMesh
checkMesh

3 DECOMPOSE THE MESH

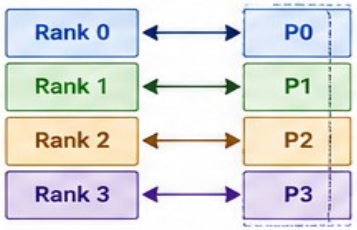
Split the mesh into
4 subdomains



decomposePar
(4 subdomains)

4 RUN IN PARALLEL (SLURM + MPI)

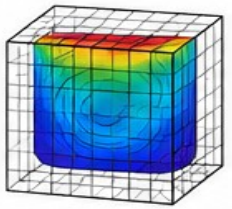
Each MPI process solves
its subdomain



mpirun / srun
icoFoam -parallel

5 RECONSTRUCT RESULTS

Combine all subdomains
into the full solution



reconstructPar

6 VIEW / ANALYZE

Visualize and analyze
the results

paraFoam
foamToVTK

7 COMPLETED

Parallel CFD
simulation finished

Results stored
on Lustre

SCIENTIFIC COMMANDS USED

1 COPY TUTORIAL CASE

```
cd /mnt/lustre/school_modeling_sim/test
cp -r $FOAM_TUTORIALS/incompressible/icoFoam/
     cavity/cavity cavity_mpi
cd cavity_mpi
```

2 CREATE MESH

```
blockMesh
checkMesh
```

3 DECOMPOSE THE MESH (4 PARTS)

```
nano system/decomposeParDict   # set n (2 2 1)
decomposePar -force
ls -d processor*
```

4 RUN IN PARALLEL WITH SLURM

```
sbatch cavity.slurm

(inside script)
mpirun -np $SLURM_NTASKS icoFoam -parallel
```

Example Slurm Resources

- `--nodes=1`
- `--ntasks=4`
- `--cpus-per-task=1`
- `--time=00:10:00`

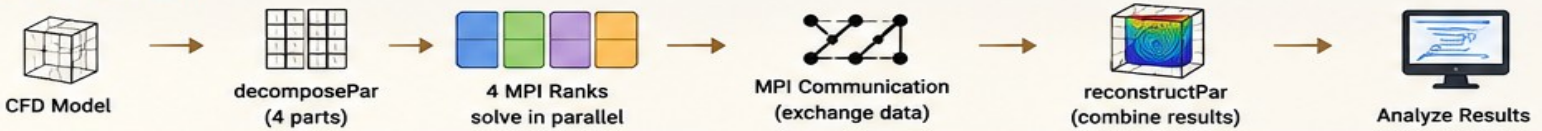
5 RECONSTRUCT RESULTS

```
reconstructPar
foamListTimes
```

6 VISUALIZE / ANALYZE

```
paraFoam
foamToVTK
```

CONCEPTUAL FLOW



HPC SYSTEM STACK

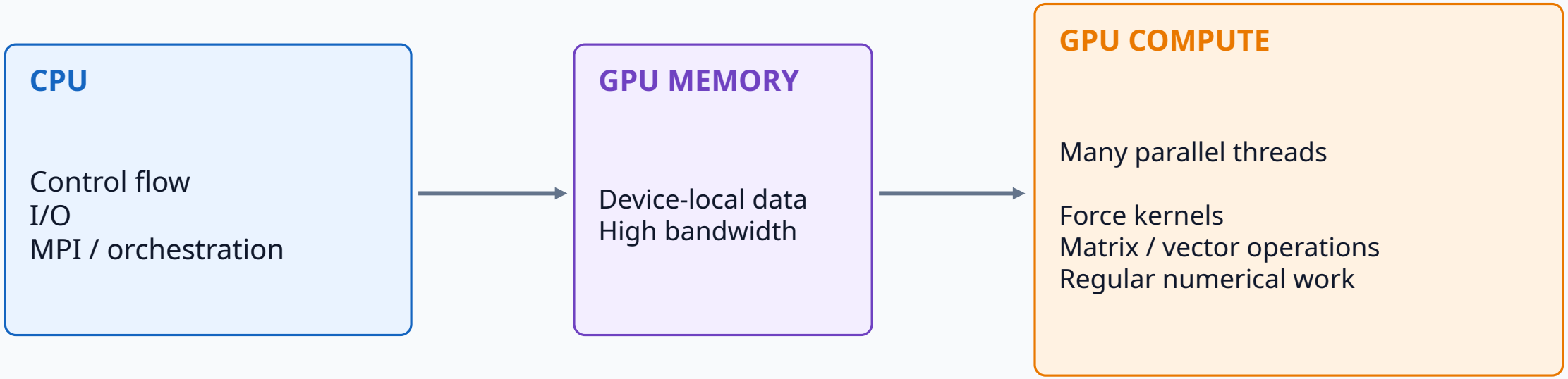
- **Slurm** — Allocates compute resources
- **OpenFOAM** — Solves CFD equations
- **MPI** — Runs multiple subdomains in parallel and communicates
- **Lustre** — Stores case files and results

KEY TAKEAWAYS

- We decomposed the CFD model into 4 subdomains.
- Slurm provided 4 MPI processes (ranks).
- Each rank solved its own part of the domain.
- MPI exchanged boundary information.
- Results were reconstructed into the full domain.
- This is the basic workflow for HPC CFD simulations!

Level 3 — GPU Acceleration

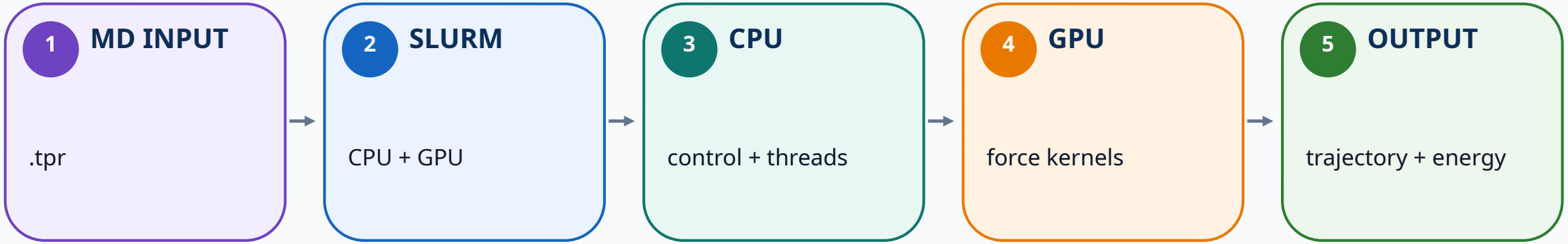
The CPU orchestrates the simulation while highly parallel numerical kernels are offloaded to the GPU.



Data movement matters: acceleration helps most when enough work stays on the GPU.

GROMACS: CPU + GPU Molecular Dynamics

A prepared molecular system is an intuitive example of heterogeneous CPU/GPU execution.



GROMACS hides most low-level GPU programming. The user chooses resources and the application maps suitable kernels to the accelerator.

Hands-On D — GROMACS with a GPU

Use a prepared small .tpr file. Focus on resource request, execution and output.

Conceptual example — adapt executable/module to cluster

```
#!/bin/bash
#SBATCH --job-name=gmx_gpu
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --gres=gpu:1
#SBATCH --time=00:20:00

module load gromacs
srun gmx mdrun -s topol.tpr
```

INPUT

Prepared topol.tpr

↓

EXECUTION

CPU orchestrates + GPU accelerates kernels

↓

OUTPUT

Trajectory + energies + log

One Cluster — Three Scientific Applications

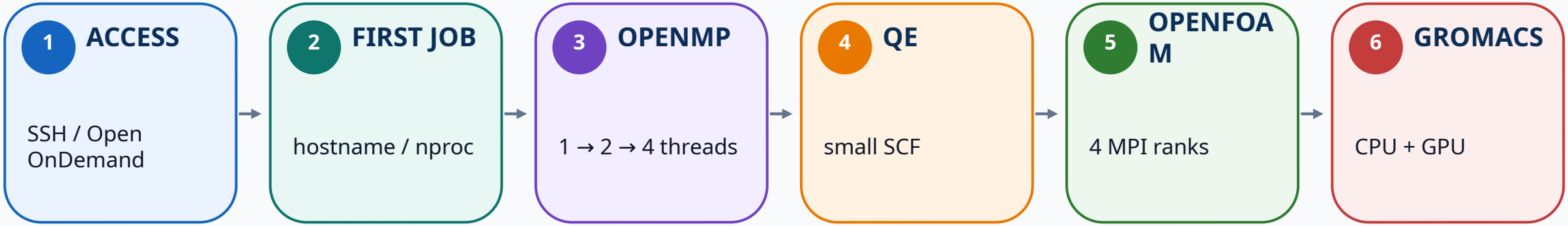
Different physics, different numerical kernels, same basic HPC workflow.

	Quantum ESPRESSO	OpenFOAM	GROMACS
Physics	Electronic structure / DFT	Fluid dynamics	Molecular dynamics
Main object	Atoms + electrons	Mesh + PDEs	Atoms + forces
Parallel focus	OpenMP / MPI	MPI decomposition	Threads / MPI / GPU
Simple hands-on	SCF energy	Cavity flow	Prepared MD run
Output	Energy / forces	Velocity / pressure	Trajectory / energy

Common workflow: input → module → Slurm → execute → monitor → results

Hands-On Roadmap

The practical session follows the same progression as the lecture.



For every exercise: start small → verify correctness → record runtime → change one resource → compare.

Good Practice for Beginner Scientific Jobs

Simple habits prevent most early HPC mistakes.

Run compute through Slurm

Do not use the login node for heavy work.

Start with small inputs

Verify the model before scaling.

Request only what you need

Extra cores/GPUs may waste capacity or reduce efficiency.

Keep scripts with results

Record modules, commands and resource requests.

Inspect logs first

Most failures are explained in stdout/stderr.

Compare before scaling

More hardware is not automatically faster.

Key Takeaways

What participants should remember before the practical session.

1. The software stack connects the scientific model to HPC hardware.
2. Users prepare files and submit jobs; Slurm allocates compute resources.
3. OpenMP uses shared-memory CPU threads within a node.
4. MPI distributes work across independent processes and nodes.
5. GPUs accelerate suitable numerical kernels; the application usually manages the low-level details.
6. **QE, OpenFOAM and GROMACS use different physics but the same basic HPC execution workflow.**

NEXT: HANDS-ON ON THE NAMAL SUPERCOMPUTING CLUSTER